

Team Document

Group 5 Members:

Hath: Manh Anh Tuan Nguyen

Conical: Marco Ariola Sanchez

Stone: Yuanhao(Stone) Ding

Philly: Philip Ngo

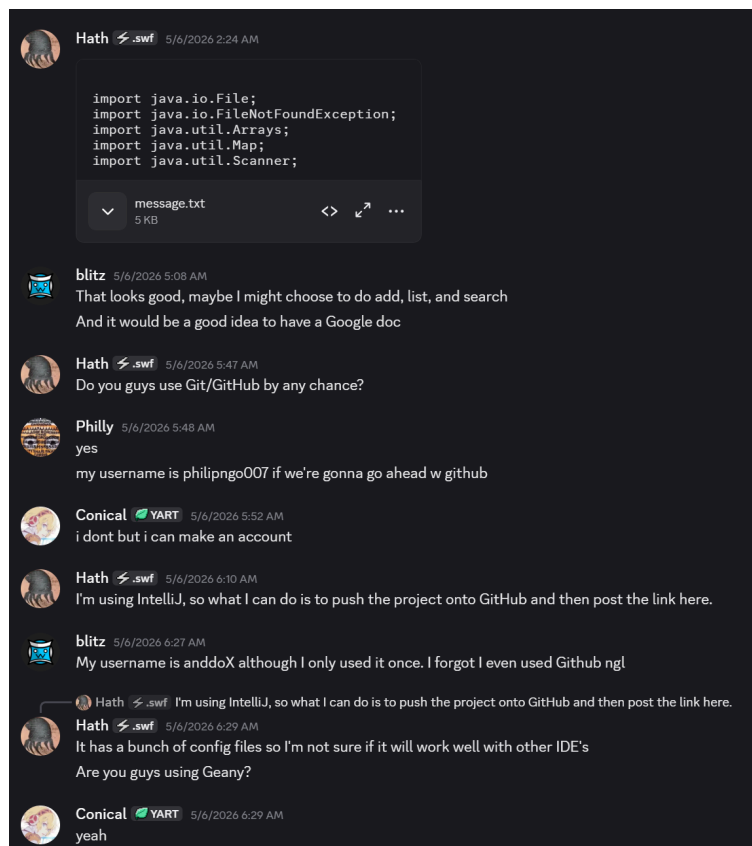
Blitz: Andrew Do

I. Timeline of the Group Project and Collaboration Record

1. Group up and start working (6/5/2026)

So our journey starts right after the second midterm. Most conversations take place on **Discord**, and the code was shared through **GitHub** and **Discord**.

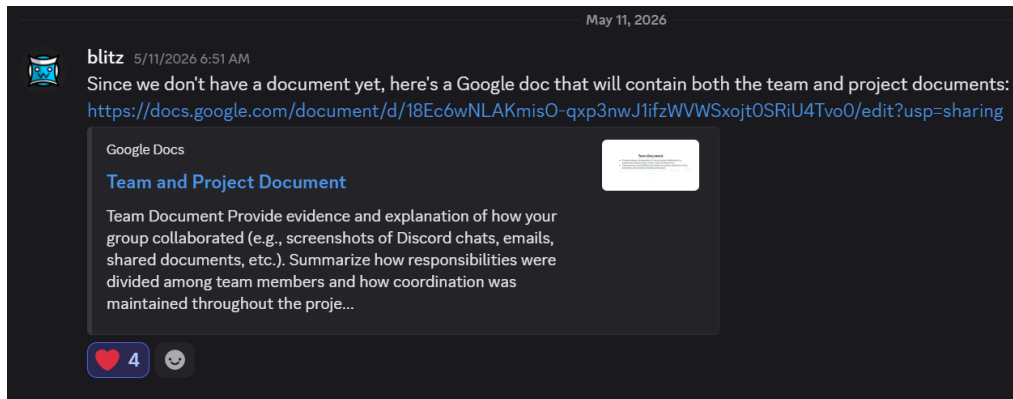
<https://github.com/harifuda/Simple-LMS>



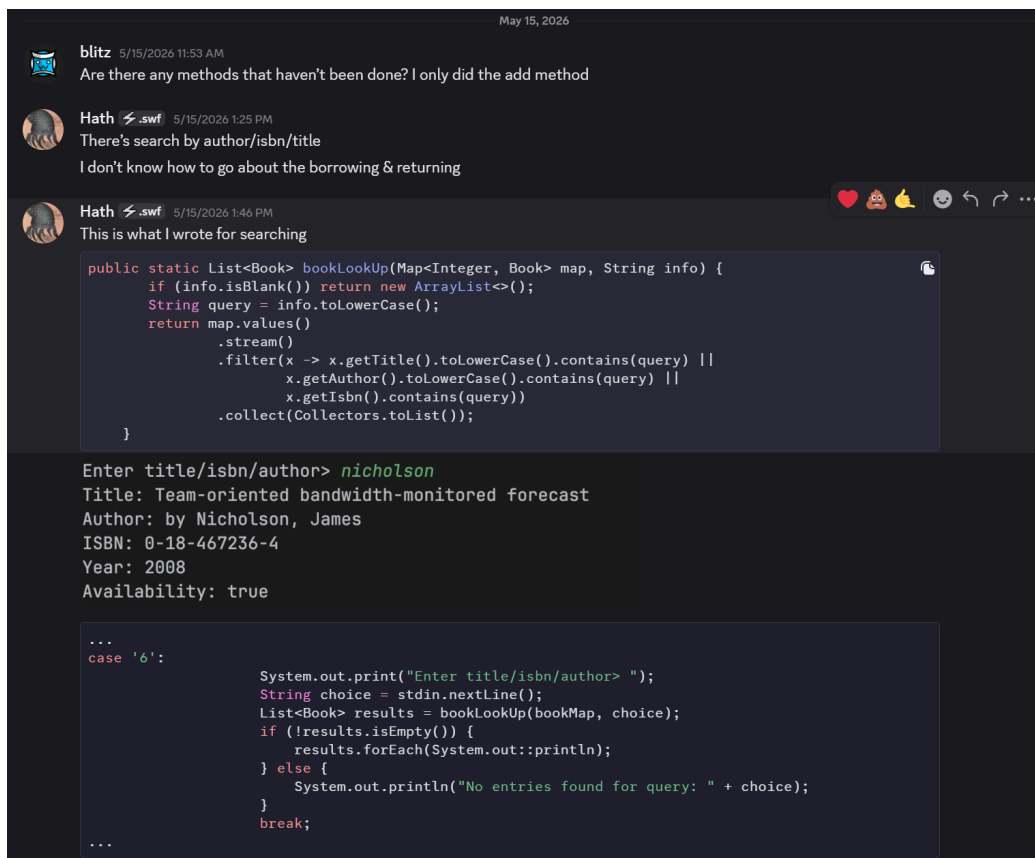
On this day, Tuan managed to finish a boilerplate of the **Library class**. After a short discussion of the IDE, we finished our first group discussion.

2. Team and Project Document created (11/05/2026)

So today, we have our documents officially created! The link to the document is right here. [Team and Project Document](https://docs.google.com/document/d/18Ec6wNLAKmisO-qxp3nwJIifzWWWSxojt0SRiU4Tvo0/edit?usp=sharing)



3. The add and search method(15/05/2026)



4. Borrow, return, and save methods are included and uploaded to Google Docs. (16/05/2026)

Hath  And the Book class `public class Book { private String title; private String author; private String isbn; ...`

Hath  5/15/2026 1:53 PM
Just realized I forgot to add a newline to the end of the toString() String

blitz 5/15/2026 2:04 PM
Ok I will try to implement borrow and maybe return

Hath  5/15/2026 2:08 PM
Awesome
I reckon it should go something like:
"User chooses book"
If book is available, set to unavailable
Otherwise, tell user book is unavailable.

2:08 PM Return's just the other way around
I think we might need a list or so to store the checked out books

May 16, 2026

Conical  5/16/2026 12:15 AM
I can implement save

Conical  5/16/2026 11:49 AM
ok i put the save method in the google doc and also did something so u can load a file since it also said that as part of the requirements on canvas

May 17, 2026

Hath  Yesterday at 12:03 AM
I'll go ahead and paste the search and load methods to the document (edited)

blitz Yesterday at 7:37 AM
I put the borrow method in the google doc, I did have to use a separate file to store the borrowed books
It probably could be more optimized but I guess it works (edited)

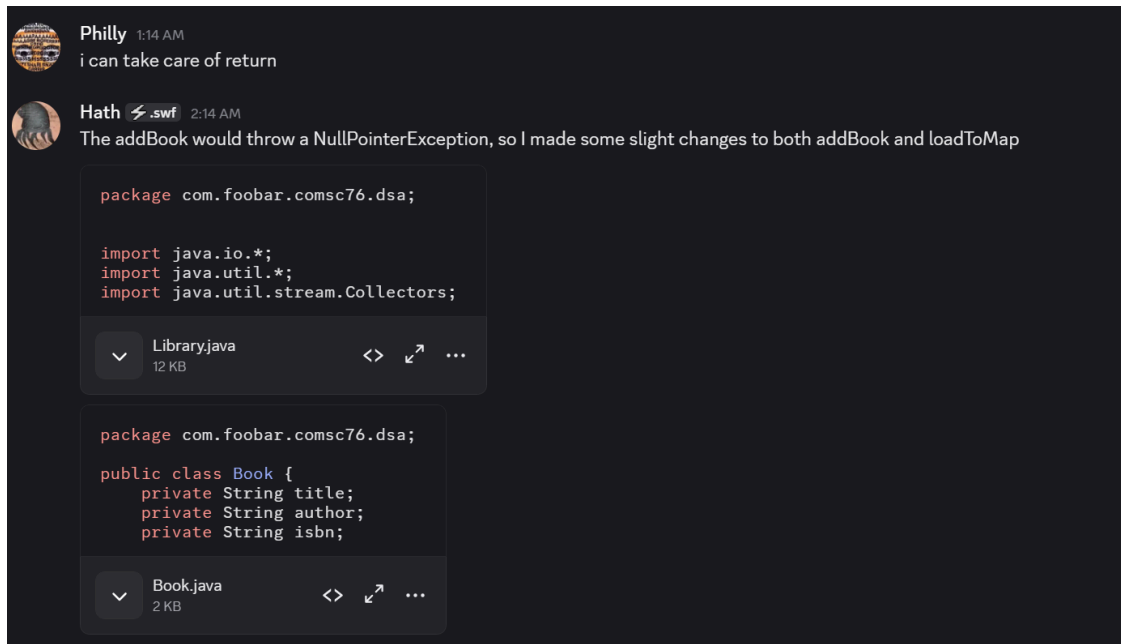
5. NullPointerException fix, Return method, and the menu. (17/05/2026)

Hath  12:57 AM
I shifted the options up a little bit, but here's what the menu looks like now

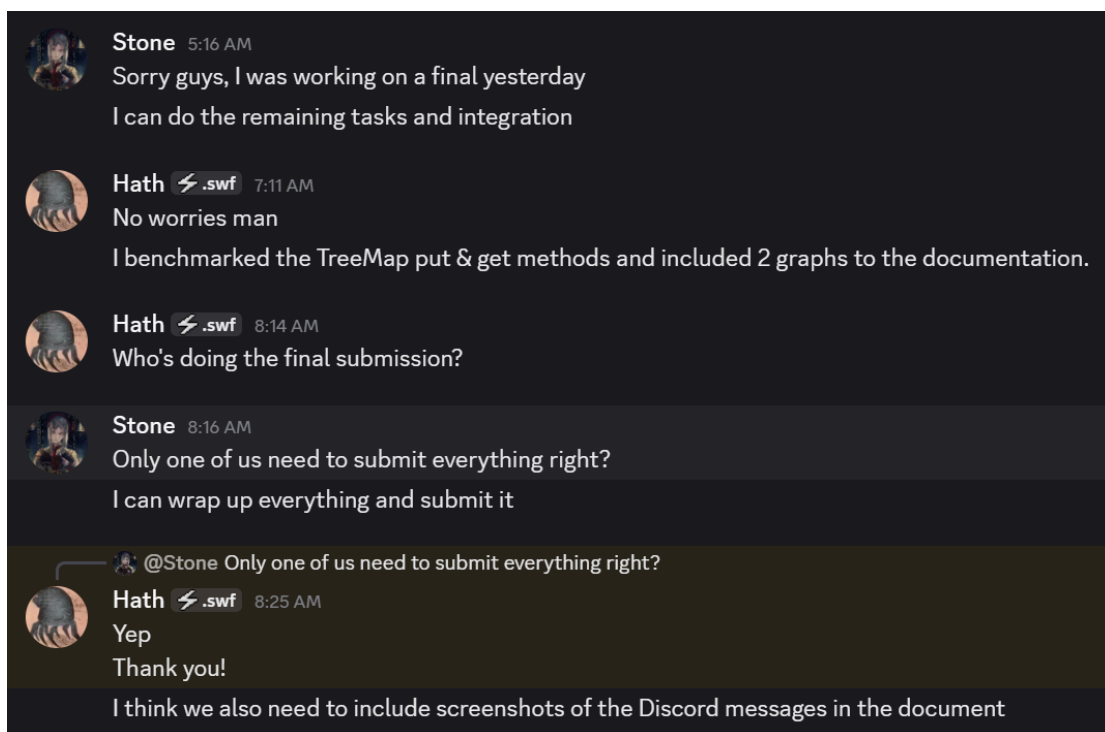
```
StringBuilder builder = new StringBuilder();
builder.append("1: List all books in the system\n");
builder.append("2: Add a book\n");
builder.append("3: Borrow a book\n");
builder.append("4: Search for a book\n");
builder.append("5: Save books to file\n");
builder.append("6: Load books from file\n");

System.out.println(builder);
```

Conical  12:59 AM
has return still not been done



6. The remaining task(team document) and the wrap-up(17/05/2026)



II. Discussion, Task Distribution, and Highlights

Interesting Discussions

Here is a discussion between Tuan and Philip on the choice of treemap or hashmap based on their efficiency, and the result will be shown in the Project Document part by **Philip**.

 **Philly** 8:34 AM
wait i wanted to mention rq
i'm playing around w using a hashmap instead of a treemap bc it reduces the time complexity down to $O(1)$ for some tasks; i have a lot of the code done alr and im gonna time it to see if it's quicker in practice or not

but if we wanna stick w the treemap that's totally fine too, i'll also help writeup a lot of the doc
if anything i can just say a pro of the treemap is the lower memory use, and i'll mention the time difference

 **Hath**  8:39 AM
Definitely!
I thought of TreeMap because it uses a Red-Black Tree for self-balancing

 1 

HashMap has the worst time complexity of $O(1)$ for put & get, so it's definitely faster in that regard (edited)

 **Philly** 10:30 AM
quick question are the times in the graphs in seconds or ms or something else
i'm gonna writeup a comparison w hashmap so i wanna make sure my units are right

10:33 AM imma eat dinner in 15ish mins, when i'm done eating i'm free to help w anything else before we submit

 **blitz** 10:35 AM
I'm guessing the time is in seconds

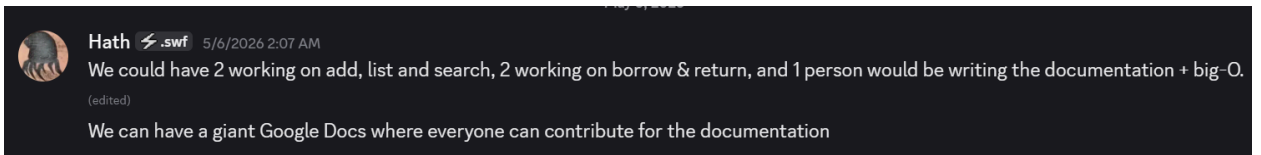
 1 

10^{-4} seconds probably seems like a reasonable amount of time for the operations

 **Hath**  10:49 AM
It's in ms, I forgot to include that in the graphs mb

 1 

The Task Distribution



Here is a rough task distribution

Tuan and Andrew: Implement methods to add and remove books and the Project Document.

Philip and Marco: Implement methods to search, borrow, return books, and the Project Document. Also justified choice of TreeMap over HashMap, an alternative

Yuanhao: Remaining tasks. Project integration. Team Documents, etc.

An emotional highlight

I really like this moment when we step out of the shadows and work like we are face-to-face offline.



Project Document

I. Code analysis

Data structure analysis

- Used a TreeMap to store data from the data file.
- TreeMap uses a Red-Black Tree for sorting, allowing for quick deletion, retrieval, and insertion due to its self-balancing nature.
- The time complexity for deletion, insertion, and retrieval is **$O(\log n)$** .
- The graphs below show the time complexity of the put() & get() method with respect to the sizes of the TreeMap.

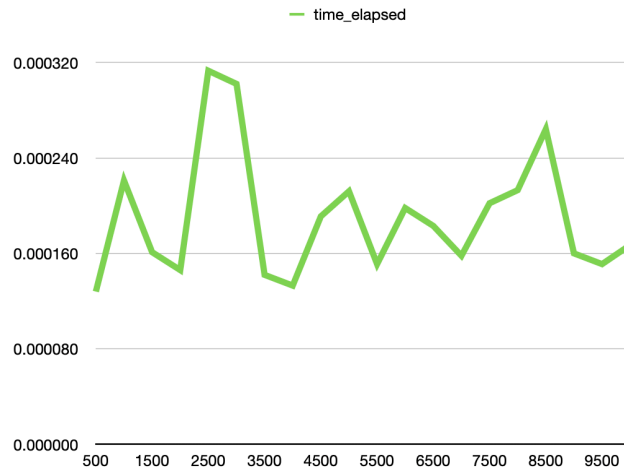
output_map_put

size	time_elapsed
500	0.000359
1000	0.000152
1500	0.000321
2000	0.000372
2500	0.000308
3000	0.000186
3500	0.000574
4000	0.000338
4500	0.000112
5000	0.000134
5500	0.000137
6000	0.000147
6500	0.000122
7000	0.000173
7500	0.000138
8000	0.000095
8500	0.000094
9000	0.000115
9500	0.000120
10000	0.000117



output_map_get_s

size	time_elapsed
500	0.000128
1000	0.000221
1500	0.000161
2000	0.000146
2500	0.000313
3000	0.000302
3500	0.000142
4000	0.000133
4500	0.000191
5000	0.000212
5500	0.000151
6000	0.000198
6500	0.000183
7000	0.000158
7500	0.000202
8000	0.000213
8500	0.000264
9000	0.000160
9500	0.000151
10000	0.000167



- Books that were borrowed were stored in an ArrayList of Book class type
- ArrayList is $O(1)$ on average for adding elements, but can be $O(N)$ only if the ArrayList gets resized
- The ArrayList uses the contains() method to mark borrowed books as false for availability, which are elements that are stored in the borrowedList
- Contains() is $O(N)$ because it looks at all the elements of the borrowedList to determine whether a book is in that list; if it is in the list, it will be considered unavailable
- **Add method, time complexity: $O(\log N)$**
 - Creates a new line and writes a new book entry at the end of the file, which is $O(1)$ since the program writes the new book entry at the last line
 - However, since the method relies on the loadToMap() method to save the added book, the time complexity on average is $O(\log N)$
- **Borrow method, time complexity: $O(\log N)$**
 - Adds the borrowed book to an arraylist which is $O(1)$ on average, but can become $O(N)$ if the arraylist must be resized
 - The borrowed book is removed from the list of books in the map, which is $O(\log N)$ on average

- Since the slower operation is the element removal from the map, $O(\log N)$ is the overall time complexity

Some Code

Adding Book

```
case '2':
    //Asking a user to add a book
    System.out.println("What is the book name?");
    title = stdin.nextLine();
    System.out.println("Who is the author? Ideally, type using
this format:");
    + "\nlastName, firstName");
    authorName = stdin.nextLine();
    System.out.println("What is the isbn of the book?");
    isbn = stdin.nextLine();
    System.out.println("Lastly, what is the publication year?");
    publicationYear = stdin.nextLine();
    addBook(bookFile, new Book(title, authorName, isbn,
publicationYear));
    break;

//Time complexity is O(N) due to the loadToMap() method
public static void addBook(File f, Book book) {
    try {
        BufferedWriter bw = new BufferedWriter(new FileWriter(f,
true));

        bw.newLine();
        bw.write(book.getTitle() + ", " + book.getAuthor() + ", " +
book.getIsbn()
        + ", " + book.getPublicYear());
        bw.close();

        loadToMap(f);
        System.out.println(book.getTitle() + " is now available.");

    } catch (IOException e) {
        System.out.println(e.getMessage());
    }
}
```

Borrowing

```
@Override
public boolean equals(Object o) {
    //Comparing fields for contains() to work
    Book book = (Book)o;
    return this.author.equals(book.author) &&
        this.title.equals(book.title) &&
        this.isbn.equals(book.isbn) &&
        this.publicYear.equals(book.publicYear);
}
```

Using an arraylist to store borrowed books, equals() is used to check whether each book is in the arraylist to mark it as borrowed, **add to the Book class**

Stored in this list, put this near the top of Library class with the bookMap:

```
public static ArrayList<Book> borrowedList = new ArrayList<>();
```

A file that contains the borrowed book to retain data

```
File borrowedBookFile = new File("borrowedBooks.txt");
try {
    borrowedBookFile.createNewFile();
} catch (IOException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
}
loadBorrowedData(borrowedBookFile);
loadToMap(bookFile);
```

Add this to the end of loadToMap, only adding available books

```
String author = builder.toString();
Book bookEntry = new Book(title, author, isbn, year);

if (borrowedList.contains(bookEntry)) {
    bookEntry.setAvailable(false);
}
else {
    bookMap.put(bookId++, bookEntry);
}
```

Basically does the same thing as loadToMap, but loads to the borrowedList instead

```
private static void loadBorrowedData(File file) {
    try (Scanner fileReader = new Scanner(file)) {
        while (fileReader.hasNext()) {
            String line = fileReader.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] pointer = line.split("\\s*,\\s*");
            if (pointer.length >= 4) {
                // Title, year, and isbn remain unchanged
                String title = pointer[0];
                String year = pointer[pointer.length - 1];
                String isbn = pointer[pointer.length - 2];
                // Authors' names range from 1 to 4
                StringBuilder builder = new StringBuilder();
                for (int x = 1; x < pointer.length - 2; x++) {
                    if (!builder.isEmpty())
                        builder.append(", ");
                    builder.append(pointer[x]);
                }
                String author = builder.toString();
                Book bookEntry = new Book(title, author, isbn, year);

                bookEntry.setAvailable(false);
                borrowedList.add(bookEntry);
            }
        }
    }

    catch (FileNotFoundException e) {
        System.out.println(e.getMessage());
    }
}
```

```
        case '3':

            int key;

            System.out.printf("Make your choice [1-%d]\n",
bookMap.size());

            try {
                key = stdin.nextInt();
                stdin.nextLine();
                System.out.println(bookMap.get(key).getTitle() + " by "
+ bookMap.get(key).getAuthor());
                borrowBook(borrowedBookFile, key);
            }
```

```

    }

    catch(Exception e) {
        System.out.println("You must type an integer only!");
        stdin.nextLine(); //to clear scanner
    }
    break;

```

```

public static void borrowBook(File f, int key) {

    Book book = bookMap.get(key);

    try {

        if(borrowedList.contains(book)) {
            System.out.println("The book is currently not
available.");
        }

        else {
            BufferedWriter bw = new BufferedWriter(new FileWriter(f,
true));

            bw.newLine();
            bw.write(book.getTitle() + ", " + book.getAuthor() + ",
" + book.getIsbn()
+ ", " + book.getPublicYear());
            borrowedList.add(bookMap.get(key));
            bookMap.remove(key);
            System.out.println("You are now borrowing " +
book.getTitle() + " by "
+ book.getAuthor());

            bw.close();
        }
    }
    catch (IOException e) {
        System.out.println(e.getMessage());
    }
}

```

Returning

```
// implementation in switch
case '4':
    if (borrowedList.isEmpty()) {
        System.out.println("No books are currently marked as borrowed.");
        break;
    }
    System.out.println("\n--- Currently Borrowed Books ---");
    for (int i = 0; i < borrowedList.size(); i++) {
        System.out.println "[" + i + "] " + borrowedList.get(i);
    }
    System.out.print("Select the index number of the book to return: ");
    try {
        int returnIndex = stdin.nextInt();
        stdin.nextLine();

        if (returnIndex >= 0 && returnIndex < borrowedList.size()) {
            returnBook(borrowedBookFile, returnIndex);
        } else {
            System.out.println("Invalid selection ID. Book index out of bounds.");
        }
    } catch (Exception e) {
        System.out.println("You must type an integer only!");
        stdin.nextLine();
    }
    break;

// method
public static void returnBook(File borrowedFile, int listIndex) {
    // put book back into library, remove from the borrowed book list
    Book bookToReturn = borrowedList.remove(listIndex);
    bookToReturn.setAvailable(true);
    bookMap.put(bookMap.size() + 1, bookToReturn);
    System.out.println("Successfully returned: " + bookToReturn.getTitle());

    try (BufferedWriter bw = new BufferedWriter(new FileWriter(borrowedFile, false))) {
        for (Book b : borrowedList) {
            bw.write(b.getTitle() + ", " + b.getAuthor() + ", " + b.getIsbn() + ", " + b.getPublicYear());
            bw.newLine();
        }
    } catch (IOException e) {
        System.out.println("Failed to rewrite tracking logs database: " + e.getMessage());
    }
}
```

Saving and Loading

```
// added to top of library class for load file option when executing
public static File currentFile;

// added options for case 7 and 8 (saving and loading)
builder.append("7: Save books to file\n");
builder.append("8: Load books from file\n");

// edited line in case 2 to make it work with load option
addBook(currentFile, new Book(title, authorName, isbn, publicationYear));

// case 7 and 8 for saving and loading
case '7':
    System.out.print("Enter filename to save to: ");
    String saveFile = stdin.nextLine();
    saveToFile(saveFile);
    break;

case '8':
    System.out.print("Enter filename to load: ");
    String loadFile = stdin.nextLine();
    File newFile = new File(loadFile);
    if (checkFileExists(loadFile)) {
        currentFile = newFile;
        loadToMap(currentFile);
    } else {
        System.out.println("File does not exist.");
    }
    break;

// added to the bottom of loadToMap method
System.out.println(
    "Books successfully loaded from "
    + file.getName());

    System.out.println(
    "Total books loaded: " + bookMap.size());

// Time Complexity: O(N), saves all books in the map to a file
public static void saveToFile(String filename) {
    try {
        BufferedWriter writer =
            new BufferedWriter(new FileWriter(filename));

        for (Map.Entry<Integer, Book> entry : bookMap.entrySet()) {
            Book book = entry.getValue();
            writer.write(
```

```
                book.getTitle() + ", " +
                book.getAuthor() + ", " +
                book.getIsbn() + ", " +
                book.getPublicYear()
            );
            writer.newLine();
        }
        writer.close();

        System.out.println("Books successfully saved to " +
filename);
    } catch (IOException e) {
        System.out.println("Error saving file: " + e.getMessage());
    }
}
```


Book search

- Asks the user for their search query, which could be the title, ISBN, or author's name of the book.
- Returns a List of all matching books.
- Displays all possible matches based on the query.
- In the event `bookLookup` returns an empty list, a message will be displayed stating that the query does not return a match.

```
public static List<Book> bookLookup(Map<Integer, Book> map, String
info) {
    if (info.isBlank()) return new ArrayList<>();
    String query = info.toLowerCase();
    return map.values()
        .stream()
        .filter(x -> x.getTitle().toLowerCase().contains(query) ||
                    x.getAuthor().toLowerCase().contains(query) ||
                    x.getIsbn().contains(query))
        .collect(Collectors.toList());
}

case '6':
    System.out.print("Enter title/isbn/author> ");
    String choice = stdin.nextLine();
    List<Book> results = bookLookup(bookMap, choice);
    if (!results.isEmpty()) {
        results.forEach(System.out::println);
    } else {
        System.out.println("No entries found for query: " + choice);
    }
    break;
```

loadToMap helper method

- Auxiliary method that reads from a file line by line and inserts into the bookMap.
- Time complexity: $O(\log n)$

```
private static void loadToMap(File file) {
    try (Scanner fileReader = new Scanner(file)) {
        int bookId = 1;
        while (fileReader.hasNext()) {
            //System.out.println(fileReader.nextLine());
            String line = fileReader.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] pointer = line.split("\\s*,\\s*");
            if (pointer.length >= 4) {
                // Title, year, and isbn remain unchanged
                String title = pointer[0];
                String year = pointer[pointer.length - 1];
                String isbn = pointer[pointer.length - 2];
                // Authors' names range from 1 to 4
                StringBuilder builder = new StringBuilder();
                for (int x = 1; x < pointer.length - 2; x++) {
                    if (!builder.isEmpty())
                        builder.append(", ");

                    //String pointerString = pointer[x];
                    //System.out.print(pointerString);
                    builder.append(pointer[x]);
                    //System.out.println(authorArray[x]);
                }
                String author = builder.toString();
                bookMap.put(bookId++, new Book(title, author, isbn,
year));

            } else {
                System.out.printf("Skipping line #%d, bad format.\n",
bookId);
            }
        }
    } catch (FileNotFoundException e) {
        System.out.println(e.getMessage());
    }
}
```

Code outputs:

1. Add a book

```
Make your selection now (ENTER to quit)>2
What is the book name?
La chasse-galerie
Who is the author? Ideally, type using this format:
lastName, firstName
Beaugrand,Honore
What is the ISBN of the book?
1-33-209704-5
Lastly, what is the publication year?
1892
Books successfully loaded from book_donation.txt
Total books loaded: 202
La chasse-galerie is now available.
```

```
200|Title: Reduced system-worthy model
Author: by Shaw, Macdonald
ISBN: 4-64-209216-6
Year: 2014
Availability: true
```

```
201|Title: Bel-ami
Author: de Maupassant, Guy
ISBN: 8-27-034475-1
Year: 1885
Availability: true
```

```
202|Title: La chasse-galerie
Author: Beaugrand, Honore
ISBN: 1-33-209704-5
Year: 1892
Availability: true
```

```
1: List all books in the system
2: Add a book
3: Borrow a book
4: Search for a book
5: Save books to file
6: Load books from file
7. Return a book
```

```
Make your selection now (ENTER to quit)>
```

2. Searching(case insensitive)

- a. By author

```
Make your selection now (ENTER to quit)>4
Enter title/isbn/author> Nicholson
Title: Team-oriented bandwidth-monitored forecast
Author: by Nicholson, James
ISBN: 0-18-467236-4
Year: 2008
Availability: true
```

- b. By ISBN

```
Make your selection now (ENTER to quit)>4
Enter title/isbn/author> 7-40-474452-5
Title: Team-oriented multi-tasking Internet solution
Author: by Campbell, Lee
ISBN: 7-40-474452-5
Year: 2018
Availability: true
```

- c. By title

```
Make your selection now (ENTER to quit)>4
Enter title/isbn/author> Bel-ami
Title: Bel-ami
Author: de Maupassant, Guy
ISBN: 8-27-034475-1
Year: 1885
Availability: true
```

d. No results

```
Make your selection now (ENTER to quit)>4
Enter title/isbn/author> laurentz
No entries found for query: laurentz
```

3. Borrowing(by ID)

```
Make your selection now (ENTER to quit)>3
Make your choice [1-202]
192
Right-sized empowering artificial intelligence by by Baker, Mcmillan
You are now borrowing Right-sized empowering artificial intelligence by by Baker, Mcmillan
```

```
192|Title: Right-sized empowering artificial intelligence
Author: by Baker, Mcmillan
ISBN: 1-50-390554-8
Year: 2021
Availability: false
```

4. Load books from file

```
Make your selection now (ENTER to quit)>6
Enter filename to load: book.txt
Books successfully loaded from book.txt
Total books loaded: 1
1: List all books in the system
2: Add a book
3: Borrow a book
4: Search for a book
5: Save books to file
6: Load books from file
7. Return a book

Make your selection now (ENTER to quit)>1
1|Title: Ulysses
Author: by James, Joyce
ISBN: 9-78-123456-5
Year: 1920
Availability: true
```

5. Returning a book

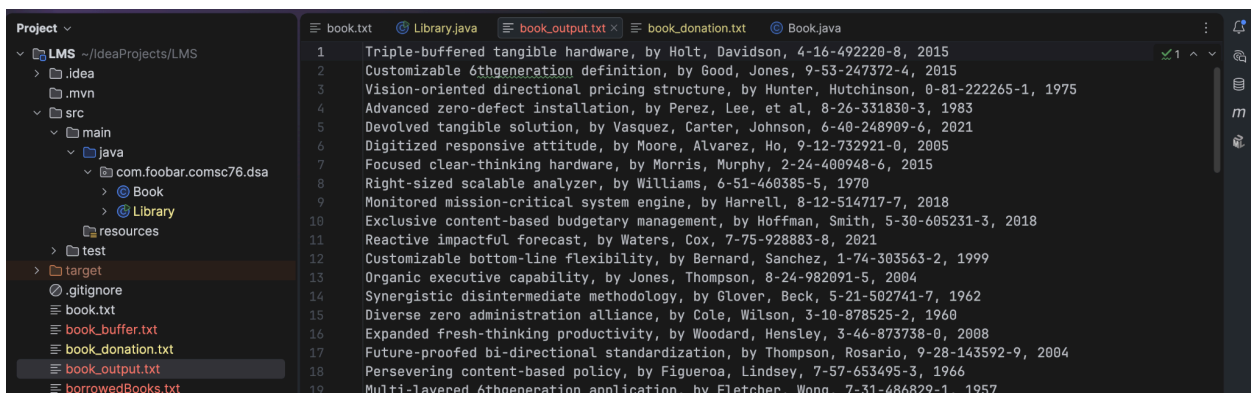
```
Make your selection now (ENTER to quit)>7

--- Currently Borrowed Books ---
[0] Title: La chasse-galerie
Author: by Honore, Beaugrand
ISBN: 1-33-209704-5
Year: 1892
Availability: true

Select the index number of the book to return: 0
Successfully returned: La chasse-galerie
```

6. Save books to a file

```
Make your selection now (ENTER to quit)>5
Enter filename to save to: book_output.txt
Books successfully saved to book_output.txt
1: List all books in the system
2: Add a book
3: Borrow a book
4: Search for a book
5: Save books to file
6: Load books from file
7. Return a book
```



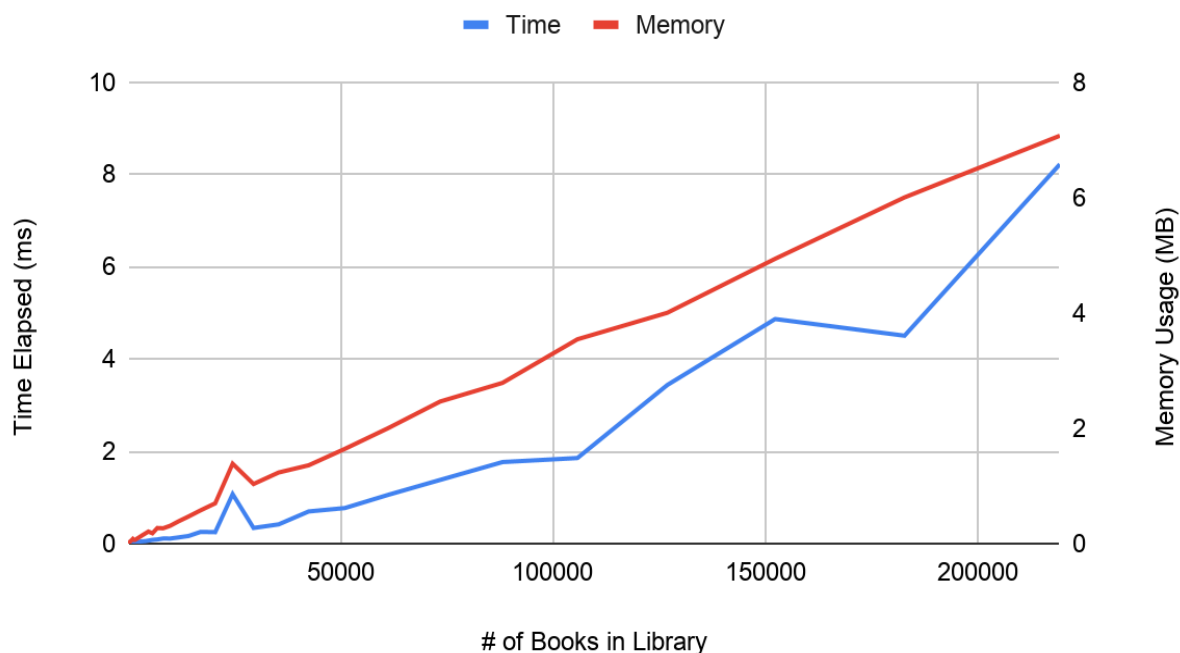
Discussion: Comparing our TreeMap & Potential HashMap

We considered using a HashMap, as the worst-case time complexity for the get and put methods for HashMaps is $O(1)$. In order to test this, we constructed a benchmark test tracking the runtime (in ms) and heap memory usage (in MB). These measurements were taken starting from 10 books and scaled to just over 200,000 books. The size of each library that was tested is given by the expression $10 * (1.2)^n < 250,000$, where n is any nonnegative number satisfying this condition. This ensures that the dataset is high-resolution.

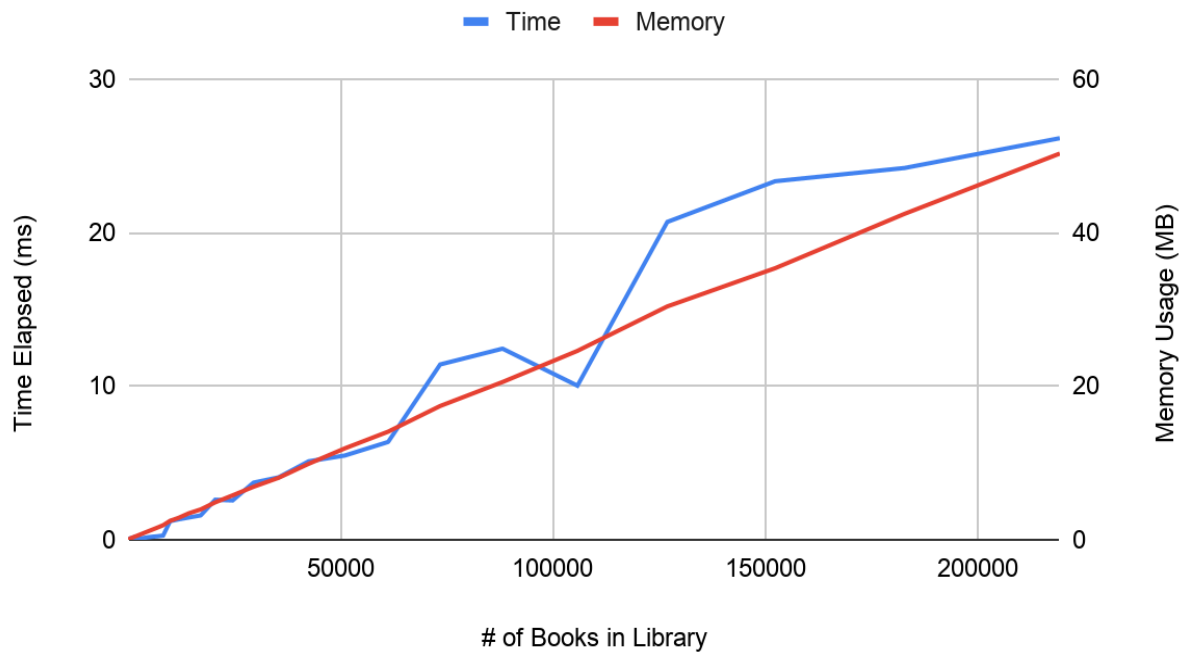
In order to account for JVM warmup time, each library size underwent 20 trials, measuring the time elapsed and memory used across each trial. We only recorded the last 5 trials for each size to let the JVM warm up; this yields more consistent results across each of the trials. An explicit garbage collection command—`runtime.gc()`—is executed right before recording the baseline memory of any given trial. This prevents data from previous iterations from altering the results of future trials.

Keeping the code as similar as possible, here is the data for the get & put methods on HashMaps:

HashMap Get Method: Time Elapsed & Memory Usage



HashMap Put Method: Time Elapsed & Memory Usage



The get method graph shows how quickly the program finds a book when it is searched for. Theoretically looking up a book should require $O(1)$ time complexity, i.e. it should take the same amount of time irrespective of the library's size. Although the time elapsed appears to increase linearly, the average time elapsed for 219,344-book libraries is only ~8.2ms, which is negligible for such a large dataset. Similarly, the average memory usage is ~7.07MB, again a relatively small amount. The reason the graph's behavior does not totally reflect $O(1)$ is due to the computer creating a random search key, i.e. an ISBN number, for each test.

Our HashMap put method benchmark also tracks time elapsed and memory usage, but this is done when adding new books to the library. The time elapsed is larger than that of the get method; it takes an average of ~26.13ms to add a book to 219,344-book libraries. This execution time is just over 3 times longer than average lookup speeds for the same size libraries. The much more significant change is the memory usage; the put method's average memory use peaks at ~55.68MB with this size of library. This has a non-negligible impact on memory use, as the program uses up more of a computer's heap memory in exchange for faster speed.

The benchmarks highlight the relative inefficiency of HashMaps' heap memory use, which has an impact on runtime in spite of the theoretical $O(1)$ performance. Observe:

- For 10,000-book libraries, the time elapsed for the TreeMap's get method is 0.000167ms. This is faster than the average time of the HashMap's get method on 9889-book libraries, which is 0.107858ms.
- For 10,000-book libraries, the time elapsed for the TreeMap's put method is 0.000117ms; the average elapsed time for 9,889-book libraries with the HashMap's put method is a much larger 1.209ms.

The increased space allocation of HashMaps is correlated with an increase in runtime. This is especially pronounced with the HashMap program's put method, even though a HashMap, on paper, should be faster with more extremely large datasets; a TreeMap's $O(\log n)$ efficiency should eventually eclipse the HashMap's $O(1)$ efficiency. Because this advantage likely only occurs for impractically large libraries and RAM is limited, we decided TreeMaps were the more practical data structure to employ.

A GitHub repo for the project using the HashMap structure is here:

<https://github.com/philipngo007/HashMapLibraryManagement>